# Auditify — Audit Engine Performance Analysis

Analysis of *Dealership_Tasks.docx* · Response-time & resource-exhaustion brief

*Prepared for: subscription@slt.work · Read-only code review — no code was modified*

## 1. What the document asks for

The brief states one problem and two proposed fixes:

- **Problem:** full audits exhaust server RAM/CPU (the site briefly goes down). You attribute it to "too many Playwright browsers."
- **Primary fix:** a user-level queue (max 5 concurrent), **2 page-workers** round-robining the 10 pages, each running the 7 sections **sequentially**, streaming each section to the client as it finishes, and saving the final report to MongoDB at the end.
- **Alternate fix:** 1 sequential page-worker + 3 section-workers.

## 2. Reality check — this reshapes the advice

The doc says *"10 workers × 8 workers each"* spawn browsers. The code does not do that. What actually happens:

- **The frontend drives the fan-out, not the backend.** `HeroSection.jsx:682` runs `Promise.all(runTargets.map(auditOnePage))` — it fires ~10–15 **separate HTTP requests at once**, one per discovered page.
- **Each request spawns a worker thread that launches its own Chromium.** `singleAuditController.js:378` → `new Worker(...)` → `puppeteer_cheerio.js:874` `chromium.launch()`. Not pooled, not semaphore-guarded.
- **The 8 sections do NOT each get a browser.** They share the worker's one browser; only Technical & Security open a couple of extra *tabs*. AIO+AEO already run sequentially first, then the other 6 in `Promise.all`.
- **There is no backend queue or global cap.** The only limiter (`concurrencyLimiter.js`) explicitly does not cover the audit workers. Multiple users multiply everything with zero ceiling.
- **Results are polling-based** (frontend polls `/status` every ~3s), not streamed.

> **True root cause**
>
> ~10–15 **unbounded, unpooled Chromium processes** per audit-click, multiplied across users — **not** 80 browsers, and **not** the 8 sections. The fix must cap the absolute number of concurrent browser processes, independent of how you structure workers.

## 3. Suggestions & opinions

### 1. Fix the ceiling, not the worker count.

The single most important change is a **global browser-concurrency semaphore** that every audit across every user draws from. "2 workers per audit" still allows 5 users × 2 = 10 concurrent browsers — no absolute cap. Size it as `server_RAM_budget ÷ ~250–350MB per headless Chromium`.

### 2. Reuse one browser; open a context/tab per page.

Launching a Chromium *process* is the expensive part. Opening a new **context** on a running browser is cheap. A shared-browser pool already exists in the main process (`puppeteer_cheerio.js:512-581`) — the workers just don't use it because they run in `worker_threads`. Biggest untapped win, and it isn't in the doc.

### 3. Reconsider worker_threads entirely.

Threads are for **CPU-bound** JS. Your heavy work is I/O-bound — Playwright (a separate OS process anyway) and PageSpeed HTTP calls (~50–75s each, which dominate wall-clock). The ~150 params are <1–80ms shared-data reads. A plain **async concurrency pool** (`p-limit`/`p-queue`) in the main process would let all pages share the pooled browser and slash RAM.

### 4. Move orchestration from frontend → backend.

Today the browser sends N page-requests. Your plan implies the *backend* owns the fan-out — the right direction, but a real refactor: consolidate to **one "audit site" request**; the backend discovers pages and fans out under its own bounded pool. A backend queue is meaningless while the frontend still fires N independent requests.

### 5. Use SSE for streaming, not a redesign.

You already accumulate per-section progress patches in the in-memory `auditStore`. Streaming section-by-section is a small delta: add one **SSE endpoint** that pushes those patches and drop the 3s polling. WebSocket is overkill.

### 6. Sequential sections = a real speed regression.

Serializing all 7 sections trades wall-clock for RAM — but sections aren't the RAM driver (browser *count* is). Better: keep cheap shared-data sections running together and serialize only the two browser/tab-heavy ones (Technical's PageSpeed, Security's active probing). Resource relief without slower pages.

### 7. Between your two approaches.

Approach A (2 page-workers, sequential sections) is more predictable than Approach B. Approach B (1 page-worker + 3 section-workers) is muddled — 3 section-workers on one page's browser means tab contention and races. But my real recommendation is **neither as written**: the global-semaphore + shared-context model bounds the absolute ceiling regardless of user count.

### 8. Two-level control is the robust design.

(a) **Admission control** — max N concurrent site-audits, rest queue (your "max 5 users"); (b) **global browser/context pool** — the hard RAM ceiling all audits share. Level (a) is fairness; level (b) is survival.

## 4. Step-by-step plan of action

1. **Measure first.** Record RAM per headless Chromium and total RAM budget; derive `MAX_CONCURRENT_BROWSER_CONTEXTS`. Capture current per-audit wall-clock as a baseline.
2. **Introduce a global browser/context pool.** One (or a few) long-lived Chromium; hand out **contexts** via a semaphore capped at the step-1 number. Replace per-page `chromium.launch()` with acquire → use → release. Extend the existing `sharedBrowser` pool.
3. **Consolidate to a single backend-orchestrated request.** Client sends the site once; backend runs discovery, then fans out pages under the pool. Deprecate the frontend per-page `Promise.all` fan-out.

4. **Add admission control (the queue).** Global cap of 5 concurrent site-audits (env-configurable); excess queues FIFO and starts as slots free. In-memory or `p-queue`; reserve BullMQ/Redis for multi-instance scale.
5. **Restructure page/section execution under the pool.** Iterate pages with bounded concurrency (start at 2, but the pool is the real limiter). Per page: run cheap shared-data sections together; serialize only Technical & Security. Keep AIO/AEO-first ordering.
6. **Stream results via SSE.** Add `GET /single-audit/:id/stream` emitting each section patch as it lands; render sections progressively on the frontend; retire polling.
7. **Persist at completion.** Keep the batched `auditStore.flush()` → `insertMany` Mongo write on `done`.
8. **Load-test & tune.** Simulate 5 concurrent audits + a 6th queued; confirm RAM/CPU stays under budget with no shutdown. Tune pool size and admission cap; verify streamed sections and the final report match.
9. **Clean up.** Retire dead per-page-request paths and unused worker-thread code if you drop threads.

## 5. A single prompt to implement this

Copy-paste the following as one instruction to an implementing agent:

> **Refactor the Auditify audit engine to eliminate server RAM/CPU exhaustion during full-site audits, while streaming results to the client section-by-section.**

> **Context (verify in code before changing anything):** Today the frontend fires ~10–15 parallel HTTP requests — one per discovered page — via `Promise.all(runTargets.map(auditOnePage))` in `Frontend/src/Component/Landing/HeroSection.jsx:682`. Each request hits `Backend/controllers/singleAuditController.js:82 (startAudit)`, which spawns a worker_thread (:378) that unconditionally calls `chromium.launch()` in `Backend/utils/puppeteer_cheerio.js:874` — a brand-new, unpooled, unbounded Chromium process per page. There is no global concurrency cap; `concurrencyLimiter.js` does not cover audit workers. The 8 pillars per page (in `Backend/workers/singleAuditWorker.js`) share the worker's one browser. Results reach the client only by ~3s polling of `/single-audit/:id/status`. A shared long-lived browser pool already exists for the main process at `puppeteer_cheerio.js:512-581` but the workers don't use it.

**Implement, in order:**

1. **Global browser-context pool + semaphore.** Extend the existing `sharedBrowser` pool so audits acquire a browser *context* (not a new process) via a semaphore capped by env var `MAX_CONCURRENT_BROWSER_CONTEXTS` (default = RAM budget ÷ ~300MB). Replace per-page `chromium.launch()` with acquire→use→release. The cap applies across ALL concurrent audits and users.
2. **Backend-owned orchestration.** Add a single "audit site" flow: client sends the site once; backend runs discovery and fans out under the pool with bounded concurrency (default 2 pages in flight; the pool semaphore is the real ceiling). Page order: Homepage → Inventory → VDP → Trade-In → Lease Specials → Service & Parts → About & Contact Us → Blog. Deprecate the frontend per-page `Promise.all` fan-out.
3. **Admission-control queue.** Cap concurrent site-audits at env var `MAX_CONCURRENT_AUDITS` (default 5); excess queues FIFO in-memory and starts as slots free. No Redis — single-instance in-memory only.

4. **Section execution.** Per page, keep cheap shared-data sections running together and serialize only the browser/tab-heavy ones (Technical/PageSpeed, Security/active-probing). Preserve AIO+AEO-first ordering. Section order: AEO & AIO → Technical → On-Page SEO → UX & Content → Accessibility → Conversion → Security.
5. **Stream via SSE.** Add `GET /single-audit/:id/stream` (Server-Sent Events) pushing each section result from `auditStore` the moment it lands; update the frontend to consume SSE and render progressively; retire the 3s polling.
6. **Persist on completion.** Keep the batched `auditStore.flush()` → `SingleAuditReport.insertMany` write when the audit finishes.

> **Prefer** an async concurrency pool in the main process over `worker_threads` unless profiling proves a CPU-bound bottleneck — the workload is I/O-bound (Playwright + ~50–75s PageSpeed calls), so threads currently just multiply browser processes for no benefit.

> **Constraints:** make all limits env-configurable with sane defaults; do not break the Mongo report schema (`Backend/models/singleAuditReport.js`) or the dedupe/section-reuse logic in `startAudit`; add structured logging of active-context and queue-depth counts. After implementing, load-test 5 concurrent audits + 1 queued and confirm RAM/CPU stays under budget with no shutdown, sections stream in, and the final report is saved intact.

## 6. Two decisions to make before implementing

**Decision A — drop worker_threads?**

**Recommended: yes.** Biggest RAM win (all pages share the pooled browser), but it's the larger refactor. Keeping threads means at least one browser process per thread.

**Decision B — accept slower pages from serial sections?**

**Recommended: no** — serialize only Technical + Security. Full serialization makes every page slower for RAM relief you can get more cheaply by capping browser count.

---

*Generated from a read-only review of the Auditify backend. No source files were modified. File references use `path:line` from branch `bugs`.*